Entwicklerhandbuch – Lernbasierte KI für Skat (Auspielen)

3. Februar 2025

Inhaltsverzeichnis

1 Einleitung **2**
- 1.1 Projektübersicht . . . 2
- 1.2 Zielgruppe . . . 2
- 1.3 Ziele dieses Handbuchs . . . 2

2 Systemübersicht und Architektur **2**
- 2.1 Gesamtsystem . . . 2
- 2.2 Architekturdiagramm . . . 2

3 Technische Voraussetzungen und Setup **3**
- 3.1 Verwendete Technologien . . . 3
- 3.2 Systemvoraussetzungen . . . 3
- 3.3 Installation und Setup . . . 3

4 Detaillierte Modulbeschreibung **3**
- 4.1 KI-Agenten(Easy, Medium, Hard) . . . 3
- 4.2 Model – Neuronales Netzwerk (`modell.py`) . . . 4
- 4.3 SkatGameEnvironment – Easy, Medium und Hard . . . 4

5 Entwicklungsprozess und Coding-Standards **5**
- 5.1 Coding-Standards . . . 5

6 Erweiterung und Anpassung **5**
- 6.1 Erweiterung des Modells . . . 5
- 6.2 Neue Spielmodi und Regelanpassungen . . . 5
- 6.3 Erweiterung der Umgebung . . . 6

7 Deployment und Betrieb **6**
- 7.1 Training und Modell-Speicherung . . . 6
- 7.2 Start des Spiels . . . 6

8 Wartung und Weiterentwicklung **6**
- 8.1 Bekannte Probleme und Verbesserungspotenzial . . . 6
- 8.2 Weiterentwicklung . . . 6

9 Anhang **6**
- 9.1 Glossar . . . 6
- 9.2 Verwendete Referenzen und Ressourcen . . . 7
- 9.3 Change Log . . . 7

1 Einleitung

1.1 Projektübersicht

Dieses Projekt implementiert drei lernbasierte KI für das Skatspiel (Easy, Medium und Hard). Der KI-Agent lernt mittels Deep-Reinforcement-Learning, Spielzüge auszuwählen und zu trainieren. Neben dem lernenden Agenten kommen auch regelbasierte KI-Gegner zum Einsatz, um realistische Spielsituationen zu simulieren.

1.2 Zielgruppe

Das Handbuch richtet sich an Entwickler und zukünftige Mitwirkende, die das Projekt verstehen, warten oder erweitern möchten. Es bietet einen Überblick über die Architektur, die Funktionsweise der einzelnen Module und Hinweise zur Erweiterung und zum Training der KI.

1.3 Ziele dieses Handbuchs

- Verständnis der Systemarchitektur und der einzelnen Komponenten
- Anleitung zum Setup und zur Inbetriebnahme des Projekts
- Erläuterung der wichtigsten Klassen und Methoden
- Hinweise zur Erweiterung, Wartung und Weiterentwicklung des Systems

2 Systemübersicht und Architektur

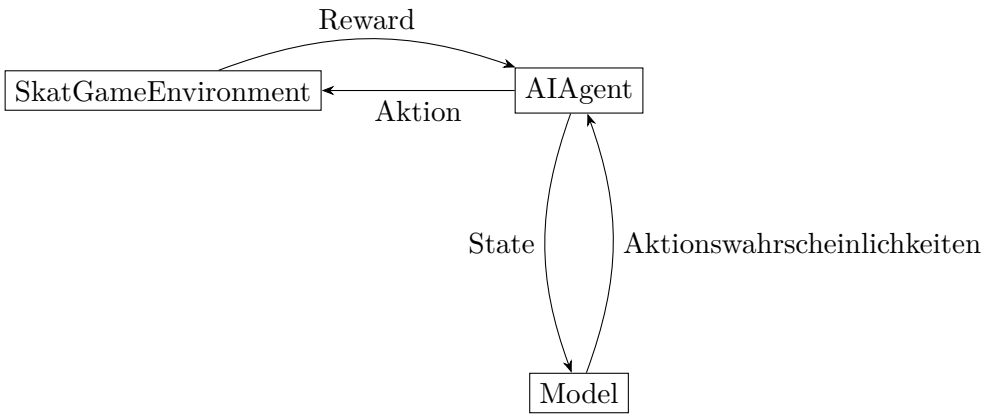
2.1 Gesamtsystem

Das System besteht aus folgenden Hauptkomponenten:

- **KI-Agenten :** Insgesamt 3 KI-Agenten für jeden Schwierigkeitsgrad. Sie sind Verantwortlich für die Entscheidungsfindung, das Training sowie das Speichern und Laden des neuronalen Netzwerks.
- **Modell (Model in `modell.py`):** Implementiert das neuronale Netzwerk, das als Schätzfunktion zur Q-Wert-Berechnung dient.
- **Spielumgebung:** Insgesamt 3 KI-Agenten für jeden Schwierigkeitsgrad. Sie Simulieren das Skatspiel, verwaltet Spielzustände, führt Züge aus, bewertet Spielrunden und berechnet Belohnungen.
- **Hilfsfunktionen (`utils.py`):** Enthält Funktionen `get_is_legal_move`, welche die Legalität eines Zuges überprüfen.

2.2 Architekturdiagramm

Das folgende Diagramm veranschaulicht den zyklischen Ablauf zwischen einer Umgebung und einem KI-Agenten und dem Modell:



3 Technische Voraussetzungen und Setup

3.1 Verwendete Technologien

- **Programmiersprache:** Python
- **Framework:** TensorFlow (für das neuronale Netzwerk)
- **Bibliotheken:** NumPy, random, collections, os

3.2 Systemvoraussetzungen

- Python 3.6 oder höher
- TensorFlow (kompatible Version)
- Weitere Bibliotheken: NumPy, etc.

3.3 Installation und Setup

1. **Abhängigkeiten installieren:** Erstelle ggf. eine virtuelle Umgebung und installiere die benötigten Pakete:

   ```
   pip install -r requirements.txt
   ```

2. **Projektstart:** Das Projekt kann über das Ausführen des Hauptskripts gestartet werden :

   ```
   python KI_agent_easy.py , python KI_agent_medium.py, python KI_agent_hard.py
   ```

4 Detaillierte Modulbeschreibung

4.1 KI-Agenten(Easy, Medium, Hard)

Der `AIAgent` ist die zentrale Komponente der lernenden KI. Die wichtigsten Funktionen im Überblick:

- **Initialisierung (`__init__`):**
  Initialisiert das Modell und lädt ggf. vorhandene Gewichte. Setzt Parameter wie `epsilon` (Explorationsrate), `epsilon_decay`, `epsilon_min`, Replay-Buffer (`memory`) und `batch_size`.

- **Aktion wählen (`act`):**
  Erzeugt aus dem aktuellen Zustand einen Zustandsvektor. Bestimmt, welche Aktionen (Karten) legal sind, indem der aktuelle Spielzustand (Hand, offene Runde, Gamemode, Turn Order) überprüft wird. Wählt per ϵ-Greedy-Strategie eine Aktion aus: Entweder zufällig (bei einem Zufallswert kleiner als ϵ) oder mittels Vorhersage des neuronalen Netzwerks (unter Beachtung einer Maske, die nicht legale Aktionen ausschließt).

- **Training (`train`):**
  Führt ein Training auf Basis eines einzelnen Übergangs (State, Action, Reward, Next State, Done) durch. Aktualisiert die Q-Werte durch Berechnung eines Targets und passt die Gewichte des neuronalen Netzes an.

- **Replay (`replay`):**
  Nutzt Experience Replay, um das Modell anhand von Zufallsstichproben aus dem Replay-Buffer zu trainieren. Aggregiert Trainingsdaten (States, Targets) und ruft anschließend die Trainingsmethode des Modells auf.

- **Modellvorhersage (`make_prediction`):**
  Berechnet mithilfe des Modells die Wahrscheinlichkeiten für alle möglichen Aktionen. Maskiert nicht legale Aktionen, sodass der Agent nur aus den zulässigen Zügen wählen kann.

- **Zustands-Vektorisierung (`_state_to_vector` und Hilfsmethoden):**

  - `_encode_hand`: Kodiert die Karten in der Hand in einen 32-dimensionalen Vektor.
  - `_encode_open_round`: Kodiert die bereits gespielten Karten in der aktuellen Runde.
  - `_encode_gamemode`: Kodiert den aktuellen Spielmodus.
  - `_encode_betting_order`: Kodiert die Reihenfolge der Spieler (Betting Order) als One-Hot-Vektor.

- **Modellspeicherung (`save_model`):**
  Speichert die aktuellen Modellgewichte in einer Datei.

4.2 Model – Neuronales Netzwerk (`modell.py`)

Das `Model`-Modul implementiert ein einfaches neuronales Netzwerk zur Vorhersage von Aktionswahrscheinlichkeiten.

- **Architektur:**
  - Eingabeschicht mit der Größe des Zustandsvektors (`state_size`).
  - Drei Dense-Schichten mit 256, 128 und 64 Neuronen (ReLU-Aktivierung).
  - Ausgabeschicht mit `action_size` Neuronen und Softmax-Aktivierung.

- **Training:** Kompiliert mit dem Adam-Optimizer und mittlerem quadratischem Fehler (MSE) als Verlustfunktion. Die Methoden `predict` und `train` sind Wrapper um die Keras-Methoden.

4.3 SkatGameEnvironment – Easy, Medium und Hard

Die Umgebung simuliert ein komplettes Skatspiel und integriert sowohl den lernbasierten Agenten als auch regelbasierte Gegner.

- **Initialisierung (`__init__`):**
  Setzt die Dealer-Instanz, definiert Gamemodes und Spieler. Initialisiert den lernbasierten KI-Spieler und instanziiert regelbasierte KI-Gegner.

- **Reset (`reset`):**
  Startet ein neues Spiel bzw. eine neue Runde. Teilt Karten aus, bestimmt die Betting Order, Skat und Gamemode und erzeugt den initialen Zustandsvektor für den KI-Agenten.

- **Zustandsbeschreibung (`_get_state`):**
  Erstellt eine Zustandsrepräsentation, die die Hand des KI-Spielers, die offene Runde, den Gamemode und die Turn Order enthält.

- **Aktionen umsetzen (`step`):**
  Führt die Aktion des KI-Agenten aus (entfernt die gespielte Karte aus der Hand). Lässt regelbasierte Gegner ihre Züge ausführen. Bewertet die Runde, indem:

  – Der Rundengewinner bestimmt wird.
  – Verschiedene Reward-Komponenten (z.B. Rundengewinn, strategische Entscheidungen, Endgame-Bewertung) berechnet werden.

  Aktualisiert den Spielzustand für die nächste Runde und gibt den neuen Zustand, die Belohnung, einen Done-Flag sowie ein Win-Flag zurück.

5 Entwicklungsprozess und Coding-Standards

5.1 Coding-Standards

- **Namenskonventionen:**

  – Klassen: CamelCase (z.B. `AIAgent`, `SkatGameEnvironment`).
  – Funktionen und Variablen: snake_case (z.B. `act`, `train`, `epsilon_decay`).

- **Dokumentation:**
  Verwende Docstrings zur Beschreibung von Klassen und Funktionen. Kommentare an kritischen Stellen erläutern Logik und Entscheidungen.

6 Erweiterung und Anpassung

6.1 Erweiterung des Modells

- **Neural Network Architektur:**
  Die Architektur in `modell.py` kann an andere Problemstellungen angepasst oder erweitert werden (z.B. Hinzufügen von Dropout-Schichten, Anpassen der Neuronenzahl).

- **Hyperparameter:**
  Parameter wie Lernrate, Batch-Größe und ϵ-Parameter können im `AIAgent` angepasst werden.

6.2 Neue Spielmodi und Regelanpassungen

- **Gamemode:**
  Erweiterungen der Gamemode-Liste in `SkatGameEnvironment` sind möglich.

- **Reward-Berechnung:**
  Änderungen in der `step`-Methode erlauben die Anpassung an neue Spielmodi.

6.3 Erweiterung der Umgebung

- **Weitere KI-Gegner:**
  Neue regelbasierte oder lernbasierte Gegner können implementiert und in die Umgebung integriert werden.

7 Deployment und Betrieb

7.1 Training und Modell-Speicherung

- **Training:**
  Das Training erfolgt während der Spielsimulationen (über die Methoden `train` und `replay` im `AIAgent`). Beachte, dass in der aktuellen Version einige Trainingsaufrufe auskommentiert sind. Diese sollten für ein produktives Training aktiviert werden.

- **Modellspeicherung:**
  Das Modell wird in regelmäßigen Abständen gespeichert (z.B. Hier alle 200 Episoden) mittels der Methode `save_model`.

7.2 Start des Spiels

- **Hauptskript:**
  Das Skript startet in der `if __name__ == "__main__`:-Sektion von jedem KI_agenten Datei. Hier erfolgt die Initialisierung der Umgebung und des Agenten sowie der Haupttrainings-/Spielschleife.

8 Wartung und Weiterentwicklung

8.1 Bekannte Probleme und Verbesserungspotenzial

- **Modellperformance:**
  Weitere Experimente mit Hyperparametern oder alternativen Netzwerkarchitekturen können die Performance verbessern.

- **Fehlerbehandlung:**
  Eine robustere Fehlerbehandlung (z.B. beim Entfernen gespielter Karten) sollte in zukünftigen Versionen integriert werden.

8.2 Weiterentwicklung

- **Neue Features:**
  Implementierung zusätzlicher KI-Gegner, erweiterte Statistiken und Visualisierungen während des Trainings.

9 Anhang

9.1 Glossar

Experience Replay

State: Der aktuelle Zustand des Spiels, bestehend aus der Hand, der offenen Runde, dem Gamemode und der Turn Order.

Action: Eine mögliche Karte, die gespielt werden kann (Index entspricht Kombination aus Farbe und Wert).

Reward: Rückmeldung, die angibt, wie gut ein Zug war.

Episode: Eine komplette Spielrunde(also 10 Rounds) bzw. Sequenz von Zügen, bis das Spiel als beendet gilt.

Experience Replay: Methode, bei der frühere Erfahrungen in zufälligen Mini-Batches zum Training genutzt werden.

9.2 Verwendete Referenzen und Ressourcen

- TensorFlow-Dokumentation: `https://www.tensorflow.org/api_docs`
- NumPy-Dokumentation: `https://numpy.org/doc/`
- Reinforcement Learning Konzepte (z.B. ϵ-Greedy, Q-Learning)

9.3 Change Log

- **Version 1.0:** Grundlegende Implementierung des lernbasierten KI-Agenten und der Spielumgebung.

Dieses Handbuch dient als Ausgangspunkt für alle Entwickler, die an der Weiterentwicklung des Skat-KI-Projekts arbeiten. Bei Fragen oder Verbesserungsvorschlägen wende dich bitte an die verantwortlichen Projektmitglieder.